Dark Forest: necesitar a los otros, temer a los otros

Aprendizaje por refuerzo multiagente en un juego estocástico parcialmente observable

Proyecto final de CC3092, Deep Learning y Sistemas Inteligentes

Equipo Dark Forest

[completar nombres, carnés y sección]

Noviembre de 2026

Resumen

Varias civilizaciones simuladas crecen en un toro bidimensional. Una anomalía cíclica las amenaza y solo puede predecirse combinando mediciones de al menos dos de ellas, pero comunicarse revela la posición y otra civilización puede atacar. Se construyó el entorno en JAX, un conjunto de estrategias fijas de referencia, un entrenamiento IPPO con GRU y una demo web que reproduce los episodios. Con estrategias fijas, cooperar aporta entre +0.2 y +3.9 de retorno según la letalidad y el tamaño de la zona (con letalidad cero no aporta nada). Con redes entrenadas, el cuello de botella fue el entrenamiento: desde cero las redes colapsan a +2.84, cerca de ignorar a las demás (+2.97), y con imitación previa de una estrategia cooperativa llegan a +4.37, sin alcanzar a esa estrategia (+5.99). De cinco hipótesis, una tiene evidencia a favor (H1), una parcial (H2) y tres no tienen apoyo o evidencia (H3, H4, H5). El documento separa lo medido de lo interpretado y declara los límites de cada cifra.

Índice

1. Descripción del problema **2**

2. Análisis **2**

2.1. Formalización 2

2.2. Lo que el análisis inicial mostró 2

2.3. Hipótesis 3

2.4. Criterios para distinguir emergencia genuina 3

3. Propuesta de solución **3**

4. Descripción de la solución **4**

4.1. Arquitectura del repositorio 4

4.2. Entorno 4

4.3. Agentes y entrenamiento 4

4.4. Evaluación 5

4.5. Demo web 5

5. Herramientas aplicadas **5**

5.1. JAX: `jit`, `vmap` y números aleatorios explícitos 5

5.2. IPPO, parámetros compartidos y GRU 6

5.3. Imitación y DAgger 6

5.4. Dinámica del replicador y masas críticas 6

5.5. Cheap talk y señales costosas 6

5.6. Entorno de cómputo: uv, Kaggle y killswitch 6

5.7. Visualización: three.js, PyVista y un sistema de diseño 7

5.8. IA generativa: prompts y evolución de la idea 7

5.9. Entendimiento de la solución 8

6. Resultados (métricas) **9**

6.1. Cómo se mide 9

6.2. Estrategias fijas 9

6.3. Redes entrenadas 10

6.4. Explotadores 11

6.5. H3: engaño y señales costosas 12

6.6. H4: coaliciones 12

6.7. Estimar N sin entrenar 12

6.8. Estado de las hipótesis 13

6.9. Pruebas del código 13

7. Conclusión **13**

1. Descripción del problema

La hipótesis del bosque oscuro de Liu Cixin [1] sostiene que, si la intención de otra civilización no puede verificarse, esconderse o atacar primero puede ser lo racional. El proyecto toma como punto de partida el escenario C de *Rogue Agents*: varios agentes reciben un desafío que ninguno resuelve solo, perciben el universo de forma parcial e individual, compiten por recursos y pueden eliminarse entre sí. Se busca saber si emergen coaliciones, conflicto, negociación o colapso.

El problema de diseño es que, en el bosque oscuro clásico, el contacto solo trae riesgo. Si el entorno se construye así, esconderse o atacar domina de forma trivial y no emerge nada: el resultado ya está en las reglas. El contrapeso que propone el proyecto es un *desafío compartido*: una anomalía periódica cuyos parámetros ocultos θ solo se identifican combinando al menos dos proyecciones ruidosas. Comunicarse vale algo, pero comunicarse revela la posición. El dilema central es:

Necesito a los otros para sobrevivir, pero los otros pueden destruirme si me encuentran.

Pregunta de investigación. ¿Cuánto debe valer la cooperación para vencer al miedo, y cómo cambia ese umbral con el retraso de las señales, la letalidad de los ataques y la posibilidad de enviar señales costosas?

Resultado esperado. Un diagrama de fases que muestre en qué condiciones aparece cada régimen: cooperación, silencio mutuo, ataque preventivo, coaliciones o colapso.

Criterio de no imposición. La recompensa de cada civilización es solo el crecimiento logarítmico de sus recursos,

$$R_{i,t} = \log(1 + r_{i,t+1}) - \log(1 + r_{i,t}), \tag{1}$$

sin términos por ocultarse, atacar o cooperar. Cualquier estrategia que aparezca debe ser instrumental. Este criterio es lo que hace que la pregunta sea científica y no un resultado de diseño.

2. Análisis

2.1. Formalización

El juego es un juego estocástico parcialmente observable (POSG) [2], $\langle N, S, \{A_i\}, \{O_i\}, T, \{R_i\}, \gamma \rangle$, con $N \in \{2, 4, 6, 8\}$ civilizaciones. Cada una ve solo su observación o_i y no conoce N ni las posiciones de las demás. Las posiciones son fijas y viven en un toro $[0, 1)^2$, es decir, un cuadrado cuyos lados opuestos se identifican; las distancias se miden por el camino más corto a través de los bordes. Hay siete acciones: crecer, ocultarse, observar, transmitir un símbolo, compartir la última medición, atacar y refugiarse. Las señales se expanden como anillos de radio $c\,(t - t_{\text{emisión}})$ y los proyectiles viajan a una velocidad menor o igual que c. Descubrir a otra civilización es que su cono de luz alcance a la propia.

2.2. Lo que el análisis inicial mostró

Antes de entrenar, se midieron estrategias escritas a mano para saber si las reglas dejaban algo que aprender (capítulo 6). Tres hallazgos cambiaron el diseño:

1. **Con periodo fijo de la anomalía, cooperar no compensa.** Una civilización sola ya despejaba la fase con su propia medición. Se sortea ahora el periodo por episodio

(parámetro `anomaly.period_spread`); es un arreglo de fidelidad, porque la propuesta ya define el periodo como parámetro oculto.

2. **Con las reglas originales, el ataque no es una decisión interesante.** Con recompensa logarítmica y crecimiento exponencial, sobrevivir 200 pasos creciendo vale cerca de 9 y morir cuesta menos de 1. Renunciar a crecer solo conviene si la probabilidad de sobrevivir creciendo es menor que cerca de 7 %.
3. **Cooperar no costaba exposición.** Crecer ya emitía en alta, así que compartir no revelaba nada nuevo. El parámetro `env.detection_growth` permite que crecer sea silencioso y que compartir sea el único acto visible.

Una consecuencia de estos hallazgos es que la hipótesis H1 se reformuló después de ver datos (sección 2.3); el cambio se declara como tal.

2.3. Hipótesis

ID	Hipótesis	Prueba
H1	Con $\lambda = 0$ la cooperación no tiene valor: anular el canal no cambia retorno ni supervivencia.	Ablación del canal
H2	Existe un λ^* desde el cual emerge la comunicación, y crece con el retraso y con p_{kill}.	Barrido $\lambda \times c$
H3	Señales caras de falsificar reducen λ^*.	Barrido con y sin señales costosas
H4	Con $N \geq 6$ y detección local aparecen grupos que comparten entre sí y callan ante los demás.	Grafo de mensajes y comunidades
H5	Cerca de λ^* unas seeds convergen a cooperación y otras a silencio.	Distribución por seed

Tabla 1: Hipótesis falsables del proyecto. λ es la letalidad de la anomalía y c la velocidad de las señales.

La redacción original de H1 decía que con $\lambda = 0$ el silencio o el ataque preventivo dominan. Se midió que crecer es la mejor respuesta a todas las estrategias probadas, así que H1 pasó a ser la prueba de cordura de que el canal no filtra información. La pregunta original se conserva como exploratoria.

2.4. Criterios para distinguir emergencia genuina

Se exigió no imposición, necesidad funcional (anular el canal y barajar los mensajes), modelo nulo (agentes aleatorios y arquetipos), robustez (cinco seeds o más) y dependencia de condiciones (barrido de parámetros).

3. Propuesta de solución

La solución tiene cuatro piezas que se apoyan una en otra:

1. **Un simulador rápido y reproducible** escrito en JAX, para entrenar miles de episodios en paralelo.
2. **Estrategias fijas de referencia** (ignorar, solo, cooperadora, recíproca, prudente, mentirosa, atacante) que actúan como modelo nulo, techo y rivales de evaluación.

3. **Agentes entrenados** con IPPO sobre una red recurrente (GRU) con parámetros compartidos, autojuego con población y, cuando el entrenamiento desde cero no converge, imitación previa de una estrategia cooperativa.
4. **Una demo web** que simula un episodio con las políticas elegidas y lo muestra, de modo que el comportamiento pueda inspeccionarse y no solo medirse.

El orden importa. Las estrategias fijas se midieron primero para decidir qué reglas dejaban algo que aprender; los agentes entrenados se compararon siempre contra esas referencias; y la demo se construyó al final, sobre los mismos episodios que produce el simulador, sin cambiar los datos.

4. Descripción de la solución

4.1. Arquitectura del repositorio

Módulo	Responsabilidad
`darkforest/`	Configuración, tipos, entorno puro (`reset`, `step`), anomalía, políticas, arquetipos y red actor-crítico GRU
`train/`	IPPO, población de autojuego, barrido, imitación, diagnósticos con políticas fijas
`analysis/`	Métricas, replicador, diagrama de fases, coaliciones, barrido entrenado
`viz/`	Animación y exportación de episodios a JSON
`demo/`	Servidor local y página web con capa narrativa

Tabla 2: Módulos y sus responsabilidades. Las dependencias van en un solo sentido: `train`, `analysis` y `viz` importan a `darkforest`, nunca al revés.

4.2. Entorno

El entorno es una función pura de estado y acciones, compatible con `jit` y `vmap`. El estado contiene posiciones, recursos, vida, nivel de emisión, parámetros ocultos θ de la anomalía y las colas de señales y proyectiles. Cada estallido de la anomalía mata, con probabilidad λ, a toda civilización dentro de la zona que no esté refugiada. Refugiarse siempre cuesta, así que no puede hacerse en todos los turnos: hay que predecir cuándo.

Se añadieron mecánicas como parámetros, apagadas o neutras por defecto, para no elegir reglas por su efecto: `anomaly.period_spread`, `env.detection_growth`, `env.detection_transmit`, `env.loot`, `env.projectiles_cancel_on_death`, y, para H3, `env.deception`, `env.costly_signals` y `env.cost_verify`.

4.3. Agentes y entrenamiento

La red es un actor-crítico con una GRU de 128 unidades, compartida entre civilizaciones y diferenciada por un identificador. Se entrena con PPO independiente por agente (IPPO). En la configuración base: 1024 entornos en paralelo, rollouts de 64 pasos, tasa de aprendizaje 3×10^{-4}, $\gamma = 0.99$, GAE con $\lambda_{\text{GAE}} = 0.95$ y recorte 0.2. El entrenamiento usa una población: con probabilidad ρ cada civilización adopta una política de la población (versiones pasadas congeladas y estrategias fijas) y no la política en entrenamiento.

Como las redes entrenadas desde cero no convergieron, se añadió un procedimiento en tres etapas: (1) clonar a la estrategia cooperativa `coop_real` mediante entropía cruzada sobre sus acciones y regresión de su valor; (2) corregir la desviación con DAgger [8]; (3) afinar con RL en población mixta. Ninguna de las tres etapas toca el entorno ni la recompensa.

4.4. Evaluación

Cada red se evalúa de dos maneras: por *muestreo* (la acción se saca de la distribución de la política) y en modo *greedy* (la acción de mayor probabilidad). Se informan las dos, porque con muestreo una probabilidad cercana a 1 % de atacar produce casi un disparo por episodio y oculta lo que la red aprendió. Las referencias (ignorar, solo, pareja `coop_real`, oráculo) se miden en el mismo mundo. Una matriz de cruce enfrenta redes y estrategias entre sí.

4.5. Demo web

La demo sirve la página y dos rutas (`/api/opciones` y `/api/simular`). Corre el simulador con el mundo y las políticas elegidos y devuelve el episodio exportado a JSON. Hay un modo científico (discos, anillos, leyenda y tabla, de referencia para comparar escenas) y un modo narrativo (sistemas estelares con planeta y astronauta, escala de Kardashev ilustrativa, flotas, ficha de civilización y bitácora). La regla es que la decoración nunca afirme algo que los datos no digan: el aspecto del planeta, el nombre del sistema y la etapa de Kardashev son decorativos; posiciones, recursos, acciones, señales, proyectiles, estallidos y muertes son los datos del episodio. Un catálogo de 12 escenarios permite presentar sin servidor.

5. Herramientas aplicadas

Se asume visto en clase: aprendizaje por refuerzo, actor-crítico y PPO, redes recurrentes y evaluación con seeds. [el equipo debe confirmar esta lista contra el temario del curso; lo que no se haya visto debe pasarse a la subsección de detalle]. Las herramientas siguientes se elaboran con más detalle.

5.1. JAX: `jit`, `vmap` y números aleatorios explícitos

JAX [9] es una biblioteca de cómputo numérico que compila funciones de Python a código para CPU o GPU mediante XLA. Tres ideas bastan para entender por qué se eligió:

- `jax.jit` compila una función una vez y la reutiliza. Por eso cada mundo nuevo obliga a recompilar el simulador (30 a 60 segundos en CPU) y los episodios siguientes del mismo mundo son rápidos.
- `jax.vmap` convierte una función de un episodio en una de un lote, sin escribir bucles. Es lo que permite simular 1024 entornos a la vez.
- La aleatoriedad es explícita: no hay estado global. Cada `PRNGKey` se pasa y se divide con `split`. Esto hace que un episodio sea reproducible dada su semilla, que es una regla del proyecto.

La contrapartida es que el entorno debe ser una función pura con formas estáticas, sin efectos secundarios; por eso se prohibió `print` y estado global en `darkforest/`. Un entorno en JAX también hace posible el trabajo en GPU: los entrenamientos finales corrieron en Kaggle con

GPU T4 (dos en paralelo). Trabajos como JaxMARL [10] usan el mismo enfoque para entornos multiagente.

5.2. IPPO, parámetros compartidos y GRU

IPPO [4] entrena a cada agente con PPO [3] tratando a los demás como parte del entorno; MAPPO [5] añade un crítico con información global. Se eligió IPPO porque cada civilización solo ve su observación y porque, con parámetros compartidos, una sola red sirve para cualquier N (se evaluó con N distinto del de entrenamiento sin reentrenar). La GRU [6] lleva una memoria que resume lo visto, necesaria porque la información es parcial y las señales llegan con retraso.

El autojuego ficticio [7] entrena contra versiones pasadas de la propia red para no quedar atrapado en un solo equilibrio. En este proyecto la población también incluye estrategias fijas (cooperadora, recíproca, atacante, solo, ignorar).

5.3. Imitación y DAgger

La clonación de comportamiento entrena una red para reproducir las acciones de un experto con entropía cruzada. Su problema es que la red, al equivocarse, visita estados que el experto nunca mostró y allí no sabe qué hacer. DAgger [8] lo corrige: la red juega, el experto etiqueta los estados que la red visita, y se vuelve a entrenar con ese conjunto ampliado. Aquí el experto es la estrategia fija `coop_real` y se usaron 4 rondas. Se añadieron pesos por rareza, porque lo que importa son pocos pasos críticos (refugiarse en el estallido) entre muchos triviales.

5.4. Dinámica del replicador y masas críticas

La dinámica del replicador [11] describe cómo cambia la fracción de cada estrategia en una población según su pago relativo. Se midió una matriz de pagos entre estrategias en el mismo simulador y se estudió hacia dónde converge una población. Como la cuenca de atracción depende de la mezcla inicial elegida, se reportó también la *masa crítica*: la fracción de una estrategia a partir de la cual vence a otra en el juego de esas dos. Con eso se construyó el diagrama de regímenes $\lambda \times c$.

5.5. Cheap talk y señales costosas

H3 viene de la teoría de señales. Un mensaje que cuesta lo mismo decir verdad que mentir no compromete a nadie [12]; una señal cuyo costo es distinto según el tipo del emisor puede separar tipos [13]. Se implementó así: con `costly_signals` el compartir verdadero cuesta más (0.06 contra 0.02) y lleva una etiqueta de verificado que la mentira no puede emitir. La comunicación emergente entre agentes de aprendizaje por refuerzo tiene su propia literatura [14, 15].

5.6. Entorno de cómputo: uv, Kaggle y killswitch

Las dependencias se fijan con `uv` (`uv.lock`); cualquier comando va con `uv run`. Los entrenamientos largos corrieron en Kaggle (GPU T4), con un *killswitch*: un interruptor dentro del kernel que lo detiene si supera las cinco horas, para no consumir el presupuesto en una corrida atascada. En la última verificación, el kernel de la corrida v10 seguía en cola sin haber empezado; sus resultados no forman parte de este documento.

5.7. Visualización: three.js, PyVista y un sistema de diseño

La vista 3D de la web usa three.js; una vista fuera de línea usa PyVista [17]. El estilo visual sale de un sistema de diseño propio (dos temas, noche y carta) con tokens y componentes, de modo que la interfaz no lleve colores sueltos. La capa narrativa se escribió como módulos independientes (astronauta, planetas, escala de Kardashev [16], efectos, bitácora, ficha de civilización) con un contrato común y pruebas en Node. La escala de Kardashev es ilustrativa: los recursos son adimensionales y la etapa no mide energía.

Una decisión física de la vista 3D: el mapa es un toro, así que las distancias y las copias al otro lado del borde se dibujan como son. El tamaño de cada sistema se reduce para no solaparse con el vecino más cercano, medido por el camino más corto a través de los bordes.

5.8. IA generativa: prompts y evolución de la idea

La herramienta fue Claude Code (Anthropic), un agente de código en la terminal, con modelos de la familia Claude [18]. También se usó un paquete de *skills* de escritura científica en formato Agent Skills [19] para estructurar este documento. Los prompts se reproducen abreviados y en el orden en que se dieron; no se corrigió su redacción.

Tabla 3: Prompts principales y qué produjeron.

Fecha	Prompt (abreviado)	Efecto
23 oct.	*"arma un esqueleto para el proyecto"*, con la propuesta completa pegada (13 000 caracteres)	Esqueleto del repositorio: configuración, entorno, pruebas
23 oct.	*"usemos uv"*	Dependencias fijadas con `uv.lock`
25 oct.	*"se paró por límites de uso, retoma donde se quedó y si terminó algún notebook de kaggle, verifícalo"*	Se retomó el trabajo y se verificó un notebook
26 oct.	*"déjame un listado de tareas para un grupo de 5 personas, yo quiero solo supervisar y hacer ajustes"*	`docs/TODO.md` y `docs/TAREAS.md`
7 nov.	*"mira qué hace falta: lineamiento, objetivo principal, idea principal, propuesta principal [...] entiendo que no converge [...] ¿dónde no estamos viendo? [...] podemos salirnos un poquito de la caja"*	Informe de estado y de lo que faltaba; plan de diagnósticos y extensiones
8 nov.	*"¿conviene probar Prime Intellect?"*, precios y *"deja un killswitch si tarda más de 5 hrs"*	Se decidió usar Kaggle como presupuesto de cómputo; se añadió el interruptor
9 nov.	*"dame next steps para después de las pruebas, qué extensiones consideradas y no consideradas podemos hacer"*	Lista de extensiones (imitación, explotadores, N grande, H3)
9 nov.	*"hacer un barrido grande que tome estas variables, ya si no funcionan se reportan y sirven como resultado"*, con lista de hipótesis	Barrido $\lambda \times c$ con redes, juego empírico y explotadores
9 nov.	*"para una visualización, ¿se va a poder ajustar políticas e inferencia sobre lo ya entrenado? [...] simular diferentes mundos"*	Servidor de simulación en vivo con redes y estrategias

Fecha	Prompt (abreviado)	Efecto
9 nov.	*"Mejora la ui con un panel en design [...] visualización 3D con pyvista o vispy"*	Sistema de diseño y visor PyVista
9 nov.	Error pegado: *"mul got incompatible shapes for broadcasting: (1, 6), (1, 2)"*	Corrección de la simulación para $N > 2$
9 nov.	*"¿hay manera de no entrenar y estimar/aproximación?"*	Evaluar redes de 2 y 6 civilizaciones con otro N sin reentrenar
9 nov.	*"no se entiende cómo crecen ni qué son galaxias o planetas [...] skin por planeta [...] escala de Kardashev [...] animar ataques con probes y flotas"*	Capa narrativa: galaxia, planetas, Kardashev, flotas
9 nov.	*"hace todo usa agentes. Quiero que tengan skins de astronauta"*	Seis módulos escritos por agentes en paralelo con un contrato común
9 nov.	*"no me termina de convencer el toro, no parece una galaxia; la animación 2D y 3D también se ven mal"*	Fondo de galaxia, nube de anomalía, tiempo fraccionario
10 nov.	*"quiero algo así"* con la ruta de un proyecto propio de sistema solar	Vista 3D estilo sistema solar: estrellas, órbitas, salto warp
10 nov.	*"ajusta cómo se muestran $n > 2$ galaxias, respetemos la física"*	Escala fiel por distancia mínima y copias al otro lado del borde

Evolución de la idea. El trabajo pasó por cuatro fases.

1. *Construir según la propuesta.* Se implementó el entorno y el entrenamiento tal como estaban especificados. El entrenamiento no convergía a "solo crecer" ni usaba el canal.
2. *Diagnosticar antes de seguir entrenando.* En lugar de entrenar más, se midieron estrategias fijas. Eso mostró que parte del problema estaba en las reglas (cooperar no costaba nada, el periodo fijo era identificable) y llevó a reformular H1 y a añadir parámetros.
3. *Atacar la optimización.* Tres cambios (minibatches, escala de recompensa, entropía decreciente) sacaron a las redes del peor colapso, y la imitación con DAgger fue la única forma encontrada de salir del colapso con refugio caro.
4. *Hacer visible el comportamiento.* La demo y su capa narrativa, iterada contra el criterio del autor: se retiraron elementos que confundían (pilares de recursos) y se reemplazaron por tamaño del disco y rótulos.

Qué falló y cómo se corrigió. Un análisis de coaliciones contaba cualquier señal recibida como mensaje; se separó el conteo de mensajes explícitos y se documentó que la conclusión depende del umbral. Una publicación de la demo como página alojada se descartó porque el autor prefirió la demo local con simulación en vivo. La vista 3D con forma de toro se reemplazó porque no se leía como galaxia. El servidor entregaba las hojas de estilo con un tipo de contenido incorrecto y la página salía sin estilo. Un escenario del catálogo con una sola política para seis civilizaciones falló hasta duplicar la política.

5.9. Entendimiento de la solución

[Esta subsección debe escribirla el equipo con su voz; el borrador siguiente es el mínimo que el equipo debería poder defender oralmente.]

El simulador es una función pura que avanza el estado un turno. El valor de comunicarse no está en la recompensa sino en la física: la anomalía solo se puede predecir juntando dos mediciones, y refugiarse a ciegas cuesta. Las redes aprenden a refugiarse al ritmo de los estallidos usando una observación pública (los turnos desde el último estallido), y con refugio barato eso basta, así que no necesitan comunicarse. Con refugio caro comunicarse rinde, pero el entrenamiento desde cero no encuentra el protocolo; la imitación sí lo aproxima. Qué no se entiende todavía: por qué la red clonada llega a entre 61 % y 94 % de la estrategia que imita y no más, y de dónde sale exactamente la ganancia de las redes afinadas sobre la heurística `solo`.

6. Resultados (métricas)

6.1. Cómo se mide

El retorno es la suma de la recompensa de cada civilización en 200 pasos. Las cifras con ± son errores estándar entre seeds. Los entrenados usan 5 seeds por configuración salvo que se indique; las estrategias fijas, 3 a 5 seeds y 100 a 400 episodios según la tabla. Cada tabla dice con qué muestra se calculó.

6.2. Estrategias fijas

El aporte de cooperar depende del periodo de la anomalía y de la letalidad. Con periodo fijo, compartir cuesta más de lo que informa (aporte real entre -0.20 y -0.24). Con periodo aleatorio el aporte real es positivo y crece con λ hasta 0.5.

Mundo	λ	`solo`	`coop_real`	Aporte real
periodo fijo	0.5	8.233	8.013	-0.219 ± 0.025
periodo aleatorio	0.25	7.764	7.946	$+0.183 \pm 0.060$
periodo aleatorio	0.5	7.272	7.765	$+0.493 \pm 0.081$
periodo aleatorio	1.0	6.954	7.456	$+0.502 \pm 0.063$

Tabla 4: Retorno medio por civilización con dos civilizaciones, 400 episodios por seed y 5 seeds. El aporte es `coop_real` menos `solo`. El techo del oráculo es 8.96.

Ablación del canal (H1 con estrategias fijas). Con $\lambda = 0$ apagar el canal cambia el retorno de `coop_real` en menos de 0.05 y con signo distinto según el radio de la zona; con $\lambda > 0$ el canal importa y su aporte crece con λ y con el radio. Las estrategias `ignorar` y `solo` rinden lo mismo con y sin canal en las ocho celdas, seed por seed: no hay fuga de información.

λ	radio	con canal	sin canal	aporte del canal
0	0.25	+8.383	+8.337	-0.682 ± 0.005
0	0.45	+8.215	+8.258	-0.850 ± 0.004
0.5	0.25	+7.917	+6.840	$+0.677 \pm 0.128$
0.5	0.45	+7.254	+3.643	$+3.543 \pm 0.153$
1	0.25	+7.778	+6.508	$+0.709 \pm 0.106$
1	0.45	+6.681	+2.503	$+3.900 \pm 0.151$

Tabla 5: Ablación del canal para `coop_real` (seis de las ocho celdas medidas; el aporte es respecto de la mejor entre ignorar y solo). Dos civilizaciones, 5 seeds, 200 episodios.

Barajar los mensajes (conservar emisor y llegada pero cambiar la medición por la de otro episodio) quita casi todo lo que quita apagar el canal: lo que vale es que el mensaje sea verdadero, no que llegue.

6.3. Redes entrenadas

Desde cero, las redes no usan el canal. Con tres cambios de optimizador (minibatches, escala de recompensa y entropía decreciente, probados juntos) cinco seeds mejoran de +1.89 a +2.39 por muestreo y a +5.49 en greedy, por encima de ignorar (+2.77) y de `solo` (+3.71), pero transmitir y compartir suman cerca de 0.4 % de las acciones. Con un refugio caro las redes colapsan: +3.5 con `cost_ref` = 0.1 y +2.9 con 0.25 o más, igual que ignorar.

La imitación sale del colapso. En el mismo mundo de refugio caro, en la celda central ($\lambda = 0.5$, $c = 0.22$):

Política		Retorno (greedy)
Ignorar		+2.97
Red desde cero (5 seeds)		+2.84
`solo` (heurística)		+4.24
Red con imitación y ajuste fino (5 seeds)	+4.37	[+4.26, +4.46]
Pareja `coop_real`		+5.99

Tabla 6: Control en la celda central. Misma población y mismo presupuesto (3.2 M pasos de ajuste) con y sin imitación previa. El intervalo es el rango entre seeds.

Barrido $\lambda \times c$ con redes (H2). La figura 1 y la tabla 7 muestran nueve celdas.

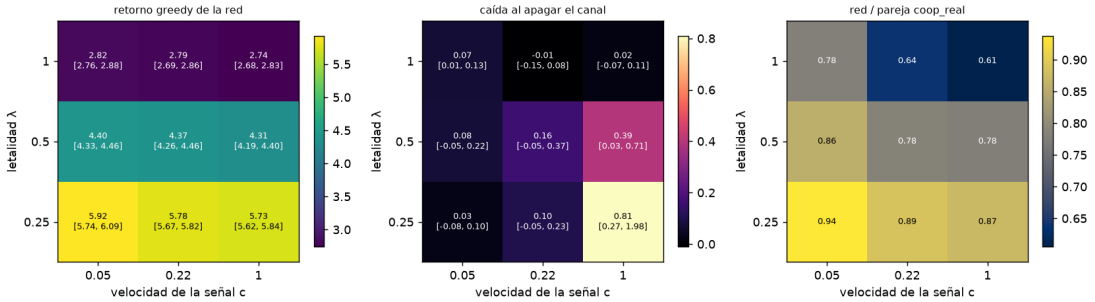


Figura 1: Barrido $\lambda \times c$ con redes entrenadas (imitación y ajuste fino, 5 seeds por celda, 200 episodios, evaluación greedy). Afirmación visual: el retorno cae con la letalidad y casi no cambia con c. Los datos de origen están en `analysis.barrido_entrenado`; la integridad cuantitativa de los ejes no se re-verificó para este documento.

λ	c	Red	Caída sin canal	`coop_real`	`solo`	Red / `coop_real`
0.25	0.05	+5.92	+0.03	+6.74	+5.97	0.94
0.25	0.22	+5.78	+0.10	+6.99	+5.97	0.89
0.25	1.0	+5.73	+0.81	+6.99	+5.97	0.87
0.5	0.05	+4.40	+0.08	+5.54	+4.24	0.86
0.5	0.22	+4.37	+0.16	+5.99	+4.24	0.78
0.5	1.0	+4.31	+0.39	+6.04	+4.24	0.78
1.0	0.05	+2.82	+0.07	+3.97	+2.85	0.78
1.0	0.22	+2.79	−0.01	+4.94	+2.85	0.64
1.0	1.0	+2.74	+0.02	+5.02	+2.85	0.61

Tabla 7: Barrido con redes entrenadas. Refugio caro (0.25), ruido 0.3, radio 0.45, 5 seeds, evaluación greedy. Las columnas de `coop_real` y `solo` salen de la matriz de cruce (200 episodios, 3 seeds de la red).

La dependencia del canal crece con la velocidad de la señal a $\lambda = 0.25$ y 0.5, que es la forma que predice H2, pero son tres valores de c, una sola configuración de mundo y redes que parten de una imitación; no se llama frontera de fase. A $\lambda = 1.0$ la red no usa el canal en ninguna celda. La red supera a `solo` solo con $\lambda = 0.5$ (por 0.07 a 0.16); con $\lambda = 0.25$ y 1.0 queda entre 0.03 y 0.24 por debajo. En ninguna celda alcanza a `coop_real`.

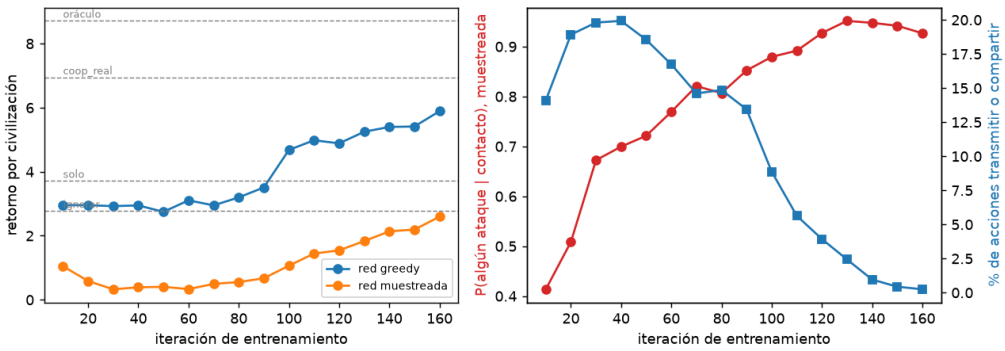


Figura 2: Evolución del aprendizaje de una seed a lo largo de 16 capturas del entrenamiento. Afirmación visual: al inicio 14 % a 20 % de las acciones son transmitir o compartir, pero apagar el canal no cambia nada (es exploración); con el entrenamiento cae a 0.2 %.

6.4. Explotadores

Cada explotador parte de su red principal y entrena 3.2 M pasos contra ella congelada. El explotador gana como mucho unos 0.2 y la principal pierde entre 0.15 y 0.26; nadie dispara. Las redes no son frágiles ante un adversario que se les ajusta, con un presupuesto de entrenamiento pequeño.

6.5. H3: engaño y señales costosas

Fila contra columna	sin señales costosas	con señales costosas
`coop_real` contra `coop_real`	+6.00	+5.75
`coop_real` contra `mentiroso`	+3.97	+3.82
`mentiroso` contra `coop_real`	+6.00	+6.00
`prudente` contra `prudente`	+3.43	+5.75
`prudente` contra `mentiroso`	+3.43	+3.27
`mentiroso` contra `prudente`	+6.04	+6.04
`solo` contra cualquiera	+3.63	+3.63

Tabla 8: Retornos con estrategias fijas, $\lambda = 0.5$, $c = 0.22$, refugio caro, ruido 0.3, 3 seeds, 200 episodios.

Sin señales costosas, el engaño paga: el mentiroso gana +6.00 contra `coop_real` y este +3.97. Con señales costosas, `prudente` coopera consigo misma (+5.75) y no se contagia de las mentiras, pero no mejora a `coop_real` frente al mentiroso y el mentiroso sigue ganando: usa las mediciones verdaderas y devuelve falsas. Con este mecanismo H3 no se sostiene: verificar baja el retorno de cooperar en 0.25 y no impide el aprovechamiento. Son estrategias fijas, sin redes entrenadas.

6.6. H4: coaliciones

Se entrenaron cinco seeds con 6 civilizaciones (3.2 M pasos, partiendo de la red clonada). Retorno greedy +4.12 [+3.96, +4.21], sin dependencia del canal. Con 12 episodios de $N = 6$ por red, supervivencia entre 0.65 y 0.68, unos 17 mensajes por episodio, 1.2 comunidades y 100 % de los mensajes dentro de la comunidad: una sola comunidad que comparte con todas. La conclusión depende del umbral de vínculo (con 3 mensajes una comunidad; con 6 se fragmenta; con 10 todas quedan aisladas), y sin un modelo nulo con la misma geometría no se puede atribuir la estructura que sí hay a algo distinto de la distancia y la velocidad de la señal. La evidencia no apoya H4.

6.7. Estimar N sin entrenar

Evaluar una red con otro número de civilizaciones sin reentrenar (40 episodios, $\lambda = 0.5$, $c = 0.22$):

Red	$N = 2$	$N = 4$	$N = 6$	$N = 8$
Entrenada con 2 civilizaciones	+4.22	+3.32	+2.52	+2.59
Entrenada con 6 civilizaciones	+4.17	+4.40	+4.22	+4.10

Tabla 9: Retorno en greedy de dos redes por familia evaluadas con distinto N sin reentrenar. Dos seeds por red; la densidad cambia con N, así que los retornos entre tamaños no son del todo comparables.

Las redes de 2 civilizaciones pierden cerca de 40 % a $N = 6$; la de 6 queda entre +4.1 y +4.4 de 2 a 8. Para $N = 4$ y 8 sirve la de 6 como estimación, pero no hay una red entrenada con ese tamaño para medir el error de la aproximación.

6.8. Estado de las hipótesis

Hip.	Estado	Razón
H1	Evidencia a favor	Con $\lambda = 0$ apagar el canal no cambia el retorno, con estrategias fijas y con redes; la muestra con redes es pequeña (5 seeds)
H2	Parcial	La dependencia del canal crece con c a λ bajo; tres valores de c, un solo mundo, redes con imitación; no es frontera de fase
H3	Sin apoyo	Verificar no impide el aprovechamiento y cuesta 0.25; sin redes entrenadas
H4	Sin apoyo	Una sola comunidad que comparte con todas; umbral y modelo nulo pendientes
H5	Sin evidencia	Sin bimodalidad entre seeds en el barrido; la imitación reduce la varianza por construcción y no la descarta

Tabla 10: Estado de las hipótesis al cierre de este documento.

6.9. Pruebas del código

El repositorio incluye pruebas automatizadas de los invariantes del entorno (recursos no negativos, muertos que no actúan, señales que viajan a velocidad c) y de las estrategias fijas y la exportación. En la última ejecución de las pruebas de la demo pasaron 9 y fallaron 2 por un error de permisos de Windows sobre la carpeta temporal de `pytest`, ajeno a la lógica probada. `ruff` no reporta errores.

7. Conclusión

El proyecto construyó un entorno donde comunicarse tiene valor funcional y costo de exposición, y midió ese valor con estrategias fijas antes de pedirles a las redes que lo descubrieran. Tres resultados sostienen la conclusión:

1. Cooperar rinde cuando la anomalía es costosa (hasta +3.9 de retorno en una celda con estrategias fijas) y no rinde cuando no lo es ($\lambda = 0$). El canal no filtra información.
2. El cuello de botella con agentes entrenados es el descubrimiento, no el valor de cooperar: desde cero las redes se quedan en +2.84, y con imitación llegan a +4.37 de un techo de +5.99.
3. Las hipótesis H3, H4 y H5 no tienen apoyo o evidencia, y H2 es parcial. Estos resultados negativos se reportan igual que los positivos.

Límites. Una sola familia de mundo para el barrido con redes, tres valores por eje, cinco seeds, redes que parten de una imitación, estrategias fijas en H3 y la mayor parte de los diagnósticos, y una corrida de $N = 6$ con 12 episodios por red. Los resultados describen este entorno y estas reglas; no son una afirmación sobre civilizaciones reales ni sobre la teoría del bosque oscuro.

Trabajo futuro. Entrenar con N grande y con señales costosas, un modelo nulo para las comunidades, un barrido con más valores de c, y repetir la imitación con otros expertos. Una corrida adicional en Kaggle (v10) quedó en cola y no forma parte de los resultados.

Repositorio. El código y este documento están en el repositorio público `https://github.com/`

`SergioAle210/RL-DarkForest`. Los comandos para reproducir cada cifra están en `docs/DIAGNOSTICO_DISENO.m` y la demo se lanza con `uv run python -m demo.servidor`.

Referencias

[1] Liu, C. (2015). *The Dark Forest* (J. Martinsen, trad.). Tor Books. (Obra original, 2008.)

[2] Hansen, E. A., Bernstein, D. S. y Zilberstein, S. (2004). Dynamic programming for partially observable stochastic games. *Proceedings of the 19th AAAI Conference on Artificial Intelligence*.

[3] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. y Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347.

[4] de Witt, C. S., Gupta, T., Makoviichuk, D., Makoviychuk, V., Torr, P. H. S., Sun, M. y Whiteson, S. (2020). Is independent learning all you need in the StarCraft multi-agent challenge? arXiv:2011.09533.

[5] Yu, C., Velu, A., Vinitsky, E., Gao, J., Wang, Y., Bayen, A. y Wu, Y. (2022). The surprising effectiveness of PPO in cooperative multi-agent games. *Advances in Neural Information Processing Systems*, 35. arXiv:2103.01955.

[6] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. y Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *EMNLP 2014*. arXiv:1406.1078.

[7] Heinrich, J. y Silver, D. (2016). Deep reinforcement learning from self-play in imperfect-information games. arXiv:1603.01121.

[8] Ross, S., Gordon, G. J. y Bagnell, J. A. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 15, 627–635.

[9] Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S. y Zhang, Q. (2018). JAX: composable transformations of Python+NumPy programs. `https://github.com/jax-ml/jax`.

[10] Rutherford, A. et al. (2023). JaxMARL: Multi-agent RL environments in JAX. arXiv:2311.10090.

[11] Taylor, P. D. y Jonker, L. B. (1978). Evolutionary stable strategies and game dynamics. *Mathematical Biosciences*, 40(1–2), 145–156.

[12] Crawford, V. P. y Sobel, J. (1982). Strategic information transmission. *Econometrica*, 50(6), 1431–1451.

[13] Spence, M. (1973). Job market signaling. *The Quarterly Journal of Economics*, 87(3), 355–374.

[14] Foerster, J. N., Assael, Y. M., de Freitas, N. y Whiteson, S. (2016). Learning to communicate with deep multi-agent reinforcement learning. *Advances in Neural Information Processing Systems*, 29. arXiv:1605.06676.

[15] Lazaridou, A. y Baroni, M. (2020). Emergent multi-agent communication in the deep learning era. arXiv:2006.02419.

[16] Kardashev, N. S. (1964). Transmission of information by extraterrestrial civilizations. *Soviet Astronomy*, 8(2), 217–221.

[17] Sullivan, C. B. y Kaszynski, A. (2019). PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK). *Journal of Open Source Software*, 4(37), 1450.

[18] Anthropic. (2026). Claude Code: documentación. `https://code.claude.com/docs`.

[19] Agent Skills. (s. f.). Especificación del formato. `https://agentskills.io/specification`.